Gráfalgoritmusok 6
Minimális feszítőfák

Szerző: Horváth Gyula
horvath@inf.elte.hu

1. Súlyozott gráfok, negatív élsúllyal

Legyen $G = (V, E, s), s : E \to \mathbb{R}$ súlyozott irányítatlan gráf. Tehát lehetnek negatív súlyú élek.
Ebben a fejezetben azt vizsgáljuk, hogy milyen feltételek mellet tudunk (hatékonyan) kiszámítani legrövidebb utakat. Először azt vizsgáljuk, hogy adott r pontra ki kell számítani minden $p \in V$ pontra a $\delta(r, p)$ $r \rightsquigarrow p$ legrövidebb út hosszát. Az alapelv az úgynevezett Bellman-Ford algoritmus. Ennek lényege hogy $n - 1$-szer hajtunk végre közelítést minden élre.

```
for (int v=1; v<=n; ++v)//inicializálás
Tav[v]=INF;
Tav[r]=0;
for(int i=1;i<n;i++){
  for (int u=1; u<=n; u++) {
    if (Tav[u]==INF) continue;
    for (auto &[v, uvsuly] : G[u]){//u->v közelítés
      if(Tav[u]+uvsuly < Tav[v])
      Tav[v]=Tav[u]+uvsuly;
    }
  }
}
```

1.1. Állítás. *Ha G-ben nincs r-ből elérhető negatív összsúlyú kör, akkor minden $p \in V$-re $\delta(r, p) = \mathtt{Tav}[p]$.*

Bizonyítás.
Tegyük fel, hogy $r = p_0, p_1, \ldots, p_k = p$ egy legrövidebb $r \rightsquigarrow p$ út. Mivel legrövidebb ut minden részútja is legrövidebb és $k < n = |V|$, ezért rendre végrehajtódik az $p_{i-1} \to p_i$ közelítés, ezért $\mathtt{Tav}[p_i] = \delta(r, p_i)$ $i = 1, \ldots, k$. Hogyan deríthető ki, hogy van-e r-ből elérhető negatív összsúlyú kör?

1.2. Állítás. *Ha $n - 1$ ciklusban végrehajtott közelítés után még egy közelítést végzünk minden élre, és $\mathtt{Tav}[u] + \mathtt{uvsuly} < \mathtt{Tav}[v]$ teljesül, akkor a v pont benne van egy negatív összsúlyú körben.*

Tegyük fel, hogy $v_0, v_1, \ldots, v_k$ egy negatív összsúlyú kör, azaz $v_0 = v_k$ és

$$\sum_{i=1}^{k} s(v_{i-1}, v_i) < 0 \tag{1}$$

Bizonyítás.
Tegyük fel, hogy $\mathtt{Tav}[v_i] \leq \mathtt{Tav}[v_{i-1}] + s(v_{i-1}, v_i)$ $i = 1, \ldots, k$-ra.
A kör mentén összegezve az egyenlőtlenségeket, azt kapjuk, hogy

$$\sum_{i=1}^{k} \mathtt{Tav}[v_i] \leq \sum_{i=1}^{k} \mathtt{Tav}[v_{i-1}] + s(v_{i-1}, v_i) = \sum_{i=1}^{k} \mathtt{Tav}[v_i] + \sum_{i=1}^{k} s(v_{i-1}, v_i)$$

Mivel $v_0 = v_k$ így

$$\sum_{i=1}^{k} \mathtt{Tav}[v_{i-1}] = \sum_{i=1}^{k} \mathtt{Tav}[v_i] \tag{2}$$

Tehát

$$0 \leq \sum_{i=1}^{k} s(v_{i-1}, v_i) \tag{3}$$

ami ellentmond az (1) egyenlőtlenségnek. Természetesen igaz, hogy ha a u pont benne van egy negatív körben, akkor nem a legrövidebb utat adja minden olyan v pontra, amely elérhető u-ból. Ezeket a pontokat egy szélességi bejárással meg tudjuk határozni.

```
#include <bits/stdc++.h>
#define maxN 100001
using namespace std;
const int INF=INT_MAX/2;
struct El{
  int pont, suly;
  El(int u, int s){
    pont=u; suly=s;
  };
};
typedef
vector<El> Graf[maxN]; //éllista
Graf G;
```

```
void Bellman_Ford(
    Graf G, int n, int r, int Tav[], int FeFa[], bool Negativ[]){
    for (int v=1; v<=n; ++v){//inicializálás
      Tav[v]=INF; FeFa[v]=-1;
    }
    Tav[r]=0; FeFa[r]=0;
    int ujtav;
    for(int i=1;i<n;i++){
      for (int u=1; u<=n; u++) {
        if (Tav[u]==INF) continue;
        for (auto &[v, uvsuly] : G[u]){
          ujtav=Tav[u]+uvsuly;
          if(ujtav<Tav[v]){
            Tav[v]=ujtav;
            FeFa[v]=u;
          }
        }
      }
    }
    //A negatív körben lévő pontok meghatározása
    queue<int> Sor;
    for (int u=1; u<=n; u++) {
      if (Tav[u]==INF) continue;
      for (auto &[v, uvsuly] : G[u]) {
        if (Tav[u]+uvsuly<Tav[v])  Sor.push(v);
      }
    }
    for (int u=1; u<=n; u++) Negativ[u]=false;
    while (!Sor.empty()) {
      int u = Sor.front(); Sor.pop();
      Negativ[u] = true;
      for (auto &[v, w] : G[u]) {
        if (Negativ[v]) continue;
        Sor.push(v);
      }
    }
  }
```

```
int  Beolvas(Graf& G, int& A){
  int n,m,u,v,s;
  cin>>n>>m;
  for(int i=0; i<m; i++){
    cin>>u>>v>>s;
    G[u].push_back(El(v,s));
    G[v].push_back(El(u,s));
  }
  return n;
}
int main(){
  int A;
  int n=Beolvas(G, A);
  int Tav[n+1];
  int FeFa[n+1];
  bool Negativ[n+1];
  Bellman_Ford(G, n, A, Tav, FeFa, Negativ);
  int B; cin>>B;
  if(Negativ[B]){
    cout<<"Negatív kör van"<<endl;
  }else if(Tav[B]==INF){
    cout<<"Nincs út "<<B<<"-be"<<endl;
  }else{
    cout<<"legrövidebb úthossz: "<<Tav[B]<<endl;
    vector<int> Ut;
    int p=B;
    while(p!=A){Ut.push_back(p); p=FeFa[p];}
    reverse(Ut.begin(), Ut.end());
    for(int p:Ut) cout<<p<<" "; cout<<endl;
  }
}
```

1.1. Gyorsítás

Látható, hogy sok esetben feleslegesen végez közelítést. Elég csak olyan pontokra elvégezni a közelítést, amelyre `Tav[u]` értéke változott. Ezt megoldhatjuk úgy, hogy egy sorba rakjuk ezeket a pontokat és onnan vesszük ki azokat, amelyekre közelíteni kell. Ha minden pontra számláljuk, hogy hányszor végeztünk rá közelítést, akkor a negatív kör jelenlétét úgy detektálhatjuk, hogy a `Kozelit[v] > n` lesz valamely `v` pontra.

```
1   #include <bits/stdc++.h>
2   #define maxN 100001
3   using namespace std;
4   const int INF=INT_MAX/2;
5
6   struct El{
7     int pont, suly;
8     El(int u, int s){
9       pont=u; suly=s;
10    };
11  };
12  typedef
13  vector<El> Graf[maxN]; //éllista
14  int  Beolvas(Graf& G, int& A, int& B){
15    int n,m,u,v,s;
16    cin>>n>>m;
17    cin>>A>>B;
18    for(int i=0; i<m; i++){
19      cin>>u>>v>>s;
20      G[u].push_back(El(v,s));
21      G[v].push_back(El(u,s));
22    }
23    return n;
24  }
```

```
bool Gyors_Bellman_Ford(Graf& G, int n, int r, int Tav[], int Fe
  int Kozelit[n+1];
  bool Sorban[n+1];
  for(int x=1;x<=n;x++){
    Tav[x]=INF;
    Sorban[x]=false;
    Kozelit[x]=0;
    FeFa[x]=-1;
  }
  queue<int> Q;

  Tav[r] = 0;
  Q.push(r);
  Sorban[r] = true;
  while (!Q.empty()){
    int u = Q.front();  Q.pop();
    Sorban[u] = false;

    for (auto [v, uvsuly] : G[u]) {
      if (Tav[u]+uvsuly < Tav[v]) {
        Tav[v]=Tav[u]+uvsuly;
        FeFa[v]=u;
        if (!Sorban[v]) {
          Q.push(v);
          Sorban[v]=true;
          Kozelit[v]++;
          if (Kozelit[v]>n)
          return false;  //negatív kör
        }
      }
    }
    return true;
  }
}
```

```
int main(){
  Graf G;
  int A,B;
  int n=Beolvas(G, A,B);
  int FeFa[n+1];
  int Tav[n+1];

  bool ok=Gyors_Bellman_Ford(G,n,A,Tav, FeFa);
  if(ok){
    cout<<Tav[B]<<endl;
    vector<int> Ut;
    for(int p=B; p!=0; p=FeFa[p])
    Ut.push_back(p);
    reverse(Ut.begin(), Ut.end());
    for(int p:Ut) cout<<p<<" "; cout<<endl;
  }else{
    cout<<"Van negatív kör"<<endl;
  }
}
```

2. Legrövidebb utak minden pontpárra, negatív élsúly esetén

Az alap Floyd-Warshall algoritmus módosítható úgy, hogy alkalmas legyen a legrövidebb utak kiszámítására olyan súlyozott gráfok esetére, amelyek tartalmazhatnak negatív súlyú éleket, de nincs a gráfban negatív összsúlyú kör.
Az elv hasonló a Bellman-Ford algoritmusnál alkalmazotthoz a negatív körök detektálására. A standard Floyd-Warshall algoritmus végrehajtása után minden `i − k − j` hármasra teszteljük, hogy ha `T[i][k]! = INF` és `T[k][j]! = INF` és `T[i][j] > T[i][k] + T[k][j]` akkor jegezzük fel, hogy a kiszámított `T[i][j]` nem helyes. Ezt megtehetjük úgy is, hogy a legrövidebb utak meghatározására érdekében kiszámított `Elso[i][j]` értékét `−1`-re állítjuk.
Ez persze nem elég, ezt a hibát propagálni kell.
Tehát ha a kiszámított `Elso[i][j] < 0`, akkor a `T[i][j]` távolság érték érvénytelem negatív kör miatt. Egyébként, ha `T[i][j] = INF`, akkor nincs `i` $\rightsquigarrow$ `j` út, egyébként van, és értéke `T[i][j]` és `Elso[][]` értékekkel megadható egy legrövidebb `i` $\rightsquigarrow$ `j` út.

```
1   #include <bits/stdc++.h>
2   #define maxN 501
3   using namespace std;
4   const int INF=INT_MAX/2;
5   typedef vector<vector<int>> Graf; //szomszédsági mátrix
6
7   void Floyd_Warshall(Graf& G, Graf& T, Graf& Elso){
8     int n=G.size()-1;
9     T.resize(n+1); Elso.resize(n+1);
10    for(int i=1;i<=n;i++){
11      T[i].resize(n+1); Elso[i].resize(n+1);
12    }
13    for (int i=1;i<=n;i++)//inicializálás, T:=G
14    for (int j=1;j<=n;j++){
15      T[i][j]=G[i][j];
16      if(i!=j && G[i][j]!=INF) Elso[i][j]=j; else Elso[i][j]=0;
17    }
18    for (int k=1;k<=n;k++){
19      for (int i=1;i<=n;i++){
20        for (int j=1;j<=n;j++){
21          int m=(T[i][k]+T[k][j]);
22          if (m<T[i][j]){
23            T[i][j]=m;
24            Elso[i][j]=Elso[i][k];
25          }
26        }
27      }
28    }
```

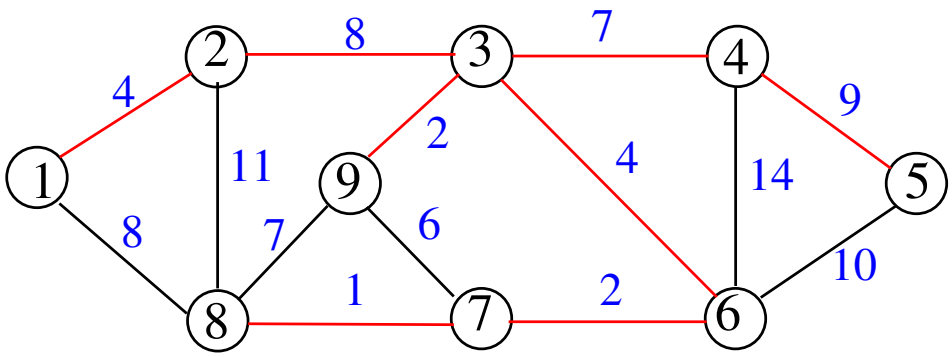
```
    //negatív körök detektálása
    for (int k = 1; k <= n; k++) {
      for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
          if (T[i][k]!=INF && T[k][j]!=INF &&
          T[i][j]>T[i][k]+T[k][j]) {
            Elso[i][j]=-1;
          }
        }
      }
    }
    //a hiba propagálása
    for (int k = 1; k <= n; k++) {
      for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
          if (T[i][k]<INF && Elso[k][j]<0) {Elso[i][j]=-1;}
          if (T[k][j]<INF && Elso[i][k]<0) {Elso[i][j]=-1;}
        }
      }
    }
  }
```

3. Minimális feszítőfák

Legyen $G = (V, E, s)$, $s : E \to \mathbb{R}^+$ súlyozott irányítatlan gráf.
Terjesszük ki a s súlyfüggvényt minden $T \subseteq E$ élhalmazra:

$$S(T) = \sum_{(u,v) \in T} s(u, v)$$

Az $F = (V, T)$ gráf minimális feszítőfája G-nek, ha
1. F feszítőfája G-nek, és
2. $S(T) \to$ minimális



1. ábra. Példa minimális feszítőfára

Input: $G = (V, E, s)$; $s : E \to \mathbb{R}^+$ irányítatlan gráf
Output: $MFF = (V, F)$; minimális feszítőfája G-nek.

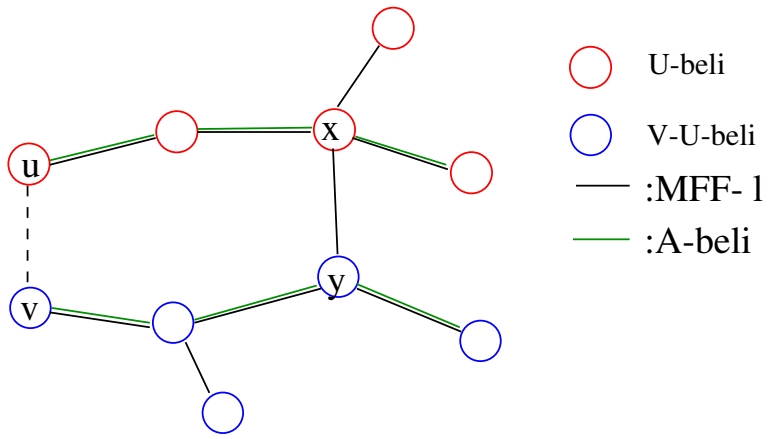
3.1. definíció. *Legyen* $A \subseteq F$ *a* G *gráf valamely* $MFF = (V, F)$ *minimális feszítőfának és* $(u, v) \in E$.
$(u, v) \in E$ **biztonságos** él A*-ra nézve, ha* $A \bigcup \{(u, v)\}$ *is része valamely minimális feszítőfának.*
A $G = (V, E)$ *gráf* vágása*:* $(U, V - U)$ *;* $U \subseteq V$*;*
$(u, v) \in E$ **keresztél** *az* $(U, V - U)$ *vágásra, ha* $u \in U$ *és* $v \in V - U$*, vagy* $u \in V - U$ *és* $v \in U$
Az $(U, V - U)$ *vágás* elkerüli *az* $A \subseteq E$ *élhalmazt, ha* A*-ban nincs keresztél.*
(u, v) **könnyű** *él az* $(U, V - U)$ *vágásra, ha a legkisebb* s*-értékű (súlyú) keresztél.*

Elvi algoritmus

$A := \emptyset$
while (A nem minimális feszítőfa) **{**
  (u, v) biztonságos él keresése;
  $A := A \bigcup \{(u, v)\}$
}

3.2. tétel. *Ha* A *része a* G = (V, E, s) *valamely minimális feszítő fájának és elkerüli az* (U, V−U) *vágást, továbbá* (u, v) *könnyű él az* (U, V − U) *vágásra, akkor* (u, v) *biztonságos él* A*-ra nézve.*

3.1. Bizonyítás



2. ábra

Legyen T = (V, F) egy olyan minimális feszítőfa, hogy A $\subseteq$ F. Ha (u, v) $\in$ F, akkor készen vagyunk. Tfh. (u, v) $\notin$ F, tehát (u, v) hozzá vétele az F faélekhez kört képez.Mivel u és v az U vágás különböző oldalán van, ezért van a körben (x, y) keresztél. Az

$$F' := F - \{(x, y)\} \cup \{(u, v)\}$$

élhalmaz is feszítőfája lesz G-nek, és s(u, v) $\leq$ s(x, y) miatt

$$C(F') := S(F) - s(x, y) + s(u, v) \leq S(F)$$

De F minimális volt, így S(F′) = S(F).

3.3. következmény. *Ha* A *része a* G = (V, E, s) *valamely minimális feszítőfának és az* (u, v) $\in$ E *élre* u *és* v *az* A *erdő két különböző fájában van és* s*-értéke (súlya) minimális, akkor az* (u, v) *él biztonságos* A*-ra.*

3.2. A Kruskal algoritmus

A biztonságos él mohó választása: mindig a legkisebb súlyú olyan élet választjuk, amely nem képez kört.

Hogyan dönthető el, hogy adott `(u, v)` él nem képez kört? Azaz ugyanazon fához tartoznak-e?

3.3. Az UnioHolvan adattípus

Az adattípus értékhalmaza valamely `n`-re az `{1, ..., n}` halmaz diszjunk részhalmazainak egy $S = \{H_1, \ldots, H_k\}$ halmaza, amelyre teljesül, hogy $\{1, \ldots, n\} = H_1 \bigcup \cdots \bigcup H_k$. Tehát minden $x \in 1..n$ -re pontosan egy olyan H_i eleme van `S`-nek, hogy $x \in H_i$

$$S = \{H_1, \ldots, H_k\} : H_i \subseteq E = \{1, \ldots, n\}, i \neq j \Rightarrow H_i \bigcap H_j = \emptyset \wedge H_1 \bigcup \cdots \bigcup H_k = E\}$$

3.4. `UnioHolvan(S, n)`

Létesít művelet, hatására $S = \{\{1\}, \ldots, \{n\}\}$

3.5. `Holvan(S, x)`

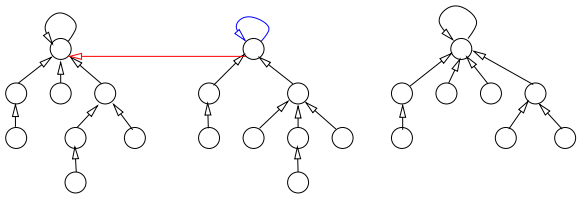
Legyen $S = \{H_1, \ldots, H_k\}$ a művelet előtt. A művelet eredmény azon H_i részhalmaz egy `Nx` eleme, a részhalmaz reprezentánsa, amelynek `x` eleme. Tehát $x \in H_i$ és $Nx \in H_i$, továbbá bármely olyan $y \in H_i$ elemre `Holvan(S, y) = Nx`.

3.6. `Unio(S, N1, N2)`

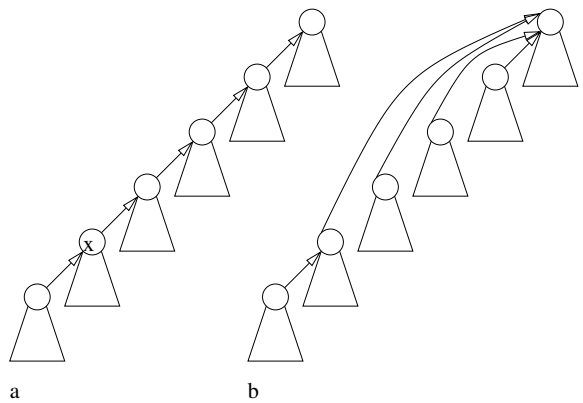
Legyen $S = \{H_1, \ldots, H_k\}$ és `N1 = Holvan(S, N1)` és `N2 = Holvan(S, N2)` és $N1 \in H_i$ és $N2 \in H_j$. Ekkor `S` művelet utáni értéke úgy kapható, hogy töröljük a halmazból a H_i és H_j elemeket, majd vegyük be új elemként a $H_i \bigcup H_j$ halmazt.

3.7. Az UnioHolvan műveletek megvalósítása

Egy $S = \{H_1, \ldots, H_k\}$ érték ábrázolásához legyen $Os : \{1, \ldots, n\} \to \{-n, \ldots, 1, \ldots, n\}$ olyan függvény, tömbbel megvalósítva, hogy minden H_i részhalmazra teljesül, hogy egy elem, az r_i reprezentáns kivételével az Os függvény fa tulajdonságú, (minden $x \in H_i$-re $Os[x] \in H_i$) és $Os[r_i] = -m$, ahol m a H_i részhalmaz elemszáma. Ezzel megvalósítható az a követelmény, hogy az `Unio` művelet mindig a nagyobbik részhalmazhoz köti a másik reprezentáns (gyökér) elemét.
Ahhoz, hogy a két művelet hatékony legyen, kel még az úgynevezett úttömörítés, ami azt jelenti, hogy minden `Holvan(x)` művelet során `x`-től a gyökérhez vezető úton lévő pontokat a gyökéhez kapcsoljuk.



3. ábra. Az Unio művelet megvalósítása halmazerdő adatszerkezettel



4. ábra. Úttömörítés a Holvan művelet során. a) a `Holvan(x)` művelet előtt, b) utána.

```
1  struct UnioHolvan{
2    vector<int> Os;
3    int Holvan(int x) {
4      if (Os[x]<0)
5      return x;
6      else
7      return Os[x] = Holvan(Os[x]);
8    }
9
10   int Unio(int Nx, int Ny){
11     if (Nx == Ny)
12     return Nx;
13     if (Os[Nx] > Os[Ny])//egyesítés a ngyobbikhoz
14     swap(Nx,Ny);
15     Os[Nx] += Os[Ny];
16     Os[Ny] = Nx;
17     return Nx;
18   }
19
20   UnioHolvan(int n){//konstruktor
21     Os=vector<int>(n+1, -1);
22   }
23 };
```

3.8. A Kruskal algoritmus negvalósítása

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  struct UnioHolvan{
4    vector<int> Os;
5    int Holvan(int x) {
6      if (Os[x]<0)
7      return x;
8      else
9      return Os[x] = Holvan(Os[x]);
10   }
11
12   int Unio(int Nx, int Ny){
13     if (Nx == Ny)
14     return Nx;
15     if (Os[Nx] > Os[Ny])//egyesítés a ngyobbikhoz
16     swap(Nx,Ny);
17     Os[Nx] += Os[Ny];
18     Os[Ny] = Nx;
19     return Nx;
20   }
21
22   UnioHolvan(int n){//konstruktor
23     Os=vector<int>(n+1, -1);
24   }
25 };
26 //
27 struct El{
28   int p,q;
29   long suly;
30   El(int x, int y, int z){p=x; q=y; suly=z;};
31   El(){}
32 };
33 typedef vector<El> Graf;
34 Graf G;
35 vector<El> FeFa;
```

```
int Beolvas(){
  int n,m,x,y,s, Nx,Ny;
  cin>>n>>m;
  for(int i=0;i<m;i++){
    cin>>x>>y>>s;
    G.push_back(El(x,y,s));
  }
  return n;
}
long long Kruskal(Graf& G, int n, vector<El>& FeFa){
  sort(G.begin(),G.end(),
          [](El e1, El e2) {return e1.suly<e2.suly;});
  UnioHolvan H=UnioHolvan(n);
  int Np,Nq;
  long long okolts=0;
  for (El e: G){
    Np = H.Holvan(e.p);
    Nq = H.Holvan(e.q);
    if (Np!=Nq){
      FeFa.push_back(e);
      H.Unio(Np,Nq);
      okolts+=e.suly;
    }
  }
  return okolts;
}
int main(){
  int n=Beolvas();
  long long okolts=Kruskal(G, n, FeFa);
  cout<<okolts<<" "<<FeFa.size()<<endl;
  for(El e:FeFa)
  cout<<e.p<<"->"<<e.q<<","<<e.suly<<endl;
}
```

3.4. tétel. *Ha a* `G` *gráf összefüggő, akkor a* ***Kruskal*** *algoritmus minimális feszítőfát állít elő.*

3.9. Bizonyítás

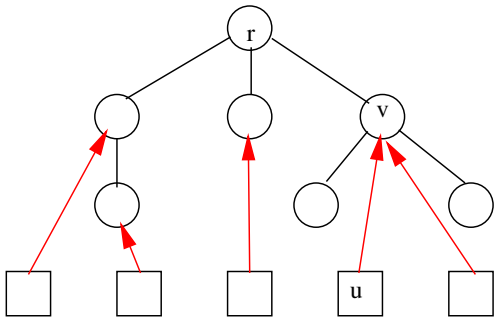
Legyen `S := {x : u` és `x` ugyanazon fában van. `}` Ekkor az `(u, v)` élre és az `(S, V − S)` vágásra teljesül a 2.2. tétel.

A Kruskal algoritmus futási ideje

Az élek rendezésének futási ideje $\mathbb{O}$`(|E| lg(|E|))`.
Az `UnioHolvan` műveletek összköltsége (amortizált költsége) a műveltek számával arányos, így az algoritmus teljes futási ideje $\mathbb{O}$`(|E| lg(|E|))`.

4. A Prim algoritmus

A biztonságos él választása: Ha `T = (S, F)` részfája `G = (V, E, c)` egy minimális feszítőfájának, és `(u, v)` egy legkisebb súlyú olyan él, hogy $u \in S$, $v \notin S$ és `c(u, v)`, akkor `(u, v)` biztonságos él.



5. ábra. A kör alakú pontok a feszítőfa kezdemény pontjai (`S`-beliek), a négyzet alakúak nem `S`-beliek, a `Q` prioritási sorban vannak. Az $u \to v$ piros él arra az `S`-beli `v` pontra mutat, amely `v` pont egy élet tekintve a legközelebb van `u`-hoz.

4.1. Megvalósítás

Adott `r` gyökerű minimális feszítőfát állítunk elő. A feszítőfa kezdemény kezdetben csak az `r` pontot tartalmazza. Vegyünk egy olyan `S`, `suly`- érték szerint minimumos prioritási sort amelyben a gráf azon `u, v, s` élei vannak, amelyekre teljesül, hogy `v` már benne van a feszítőfa kezdeményben, de `u` még nincs. Tehát minden lépésben a legkisebb súlyú élet választjuk, amely hozzávehető az építendő feszítőfához.

```
1  #include <bits/stdc++.h>
2  #define maxN 10001
3  using namespace std;
4
5  struct El{
6    int pont, suly;
7    El(int u, int s){
8      pont=u; suly=s;
9    };
10   El(){};
11 };
12 typedef
13 vector<El> Graf[maxN]; //éllista
14 Graf G;
15 El FeFa[maxN];
```

```
int Beolvas(){
  int n, m,u,v; int s;
  cin>>n>>m;
  for(int i=0;i<m;i++){
    cin>>u>>v>>s;
    G[u].push_back({v,s});
    G[v].push_back({u,s});
  }
  return n;
}

int Prim(Graf& G, int n, int r, El FeFa[]) {
  //Kimenet: FeFa; r győkerű minimális feszítőfa
  //rendezési reláció a prioritási sorhoz:
  auto rendez=[](El x,  El y){return x.suly>y.suly ||
              x.suly==y.suly && x.pont>y.pont;};
  priority_queue<El, vector<El>, decltype(rendez)> Q(rendez);
  int Tav[n+1];
  long long Kolts=0;
  int INF=INT_MAX/2;
  for(int i=1;i<=n;i++) Tav[i]=INF;
  Tav[r]=0;
  Q.push({r,0});

  while (!Q.empty()) {
    auto [u, uTav]=Q.top(); Q.pop();
    if(Tav[u]<uTav) continue;
    Kolts+=uTav;
    Tav[u]=0;//u már FeFa-ban van
    for (auto [v, uvsuly]:G[u]){//finomítás u->v élre
      if(Tav[v]==0) continue;//v már a FeFa-ban van
      if(uvsuly<Tav[v]){//v közelebb került FaFa-hoz
        Q.push({v,Tav[v]=uvsuly});
        FeFa[v]={u,uvsuly};
      }
    }
  }
  return Kolts;
}
```

```
int main(){
  int n=Beolvas();
  long kolts=Prim(G, n, 1, FeFa);
  cout<<kolts<<endl;
  for(int u=2;u<=n;u++)
    cout<<u<<"->"<<FeFa[u].pont<<":"<<FeFa[u].suly<<endl;
}
```

4.2. A Prim algoritmus futási ideje

$\mathbb{O}$(|E|.log(|E|))

5. Feladat1: mester/feszítőfák:9

Villamosítás

Kukutyin polgármestere elhatározta, hogy bevezeti a villamosközlekedést városába. A célja, hogy a város bármely útkereszteződéséből bármely másik útkereszteződésbe el lehessen jutni villamossal közvetlenül vagy átszállásokkal. Ismeri minden útszakaszra a villamosjárat kiépítésének költségét, a villamoshálózat kiépítését azonban a lehető legkevesebb pénzből szeretné megoldani. Szeretné azt is tudni, hogy melyek azok az útszakaszok, amelyek szerepelhetnek egy legkisebb költségű megoldásban.

Készíts programot, amely megadja a legkisebb összköltséget, és azt, hogy mely útszakaszok jöhetnek szóba a villamosjáratok kiépítéséhez!

Bemenet

A `standard bemenet` első sorában az útkereszteződések száma ($2 \le N \le 10\,000$) és az útszakaszok száma szerepel ($1 \le M \le 500\,000$). A következő M sorban egy-egy útszakasz két végpontján levő kereszteződés sorszáma ($1 \le A_i \ne B_i \le N$) és a szakaszon a kiépítés költsége ($1 \le K_i \le 1000$) van.

Kimenet

A `standard kimenet` első sorába a legkisebb összköltséget kell írni, amiből a hálózat kiépíthető! A második sorba azon útszakaszok U száma kerüljön, amelyek szóba jöhetnek a hálózat kiépítésénél! A harmadik sor U egész számot tartalmazzon növekvő sorrendben, azon útszakaszok sorszámait a bemenet sorrendjében, amelyek szóba jöhetnek a hálózatépítés során!

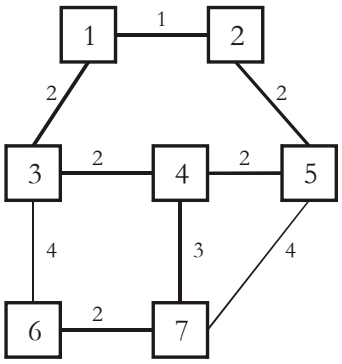
Példa

Bemenet

```
7 9
1 2 1
1 3 2
2 5 2
3 4 2
5 4 2
5 7 4
7 4 3
6 7 2
3 6 4
```

Kimenet

```
12
7
1 2 3 4 5 7 8
```



Korlátok

Időlimit: 0.1 mp.

Memórialimit: 32 MiB

Pontozás: A tesztek 30%-ában a $N \le 100$

6. Feladat2: mester/feszítőfák:12

MobilNet

A RektiNet mobilszolgáltató N átjátszó állomást telepített. Ismerjük mindegyik helyét a koordinátáikkal megadva, melyek pozitív egész értékek. Minden állomás teljesítménye egy pozitív egész számmal jellemezhető. Az `(a.x,a.y)` és a `(b.x,b.y)` helyeken lévő két állomás pontosan akkor tud egymással **közvetlenül** (oda-vissza) **kommunikálni**, ha van olyan `R` egész szám, hogy

- mindkettő teljesítménye legalább `R`, és
- `a.x=b.x` és `|a.y-b.y|`≤`R`, vagy `a.y=b.y` és `|a.x-b.x|`≤`R`.

A jelenleg működő hálózatban olyanok az állomások teljesítményei (nem feltétlenül azonosak), hogy bármely két állomás – esetleg több más állomáson keresztül – tud kommunikálni egymással.

A szolgáltató minden állomásra új kommunikációs berendezést kíván telepíteni. Az állomások új teljesítményét úgy szeretnék megválasztani, hogy továbbra is bármely kettő tudjon egymással kommunikálni. Szeretnék meghatározni, hogy mi az a legkisebb `T` teljesítmény, amelyre teljesül, hogy ha minden állomás teljesítménye `T` lesz, akkor biztosan tud kommunikálni bármely két állomás. A szolgáltató azt is tudni akarja, hogy ha nem feltétlenül azonos teljesítményű lenne minden állomás, akkor legkevesebb hány olyan `(U,V)` állomáspárnak kell lennie, hogy `U` és `V` tud egymással közvetlenül kommunikálni és `U` is és `V` is a maximális `T` teljesítményű, ahhoz, hogy bármely két állomás tudjon egymással kommunikálni.

Írj programot, mely megadja a választt a szolgáltató kérdéseire!

Bemenet

A `standard bemenet` első sorában az átjátszó állomások száma (2≤`N`≤`100 000`) van. A következő `N` sor egy-egy állomás koordinátáit tartalmazza (1≤x_i,y_i≤`10 000 000`). Minden helyen legfeljebb egy állomás van.

Kimenet

A `standard kimenet` első sorába azt a legkisebb `T` egész számot kell írni, amelyre teljesül, hogy ha minden állomás teljesítménye `T`, akkor bármely kettő tud egymással kommunikálni (esetleg közbülső állomások alkalmazásával)! A második sorba azoknak a közvetlenül kommunikáló állomáspároknak a minimális `K` számát kell írni, ahol mindkét állomásnak `T` teljesítményűnek kell lennie, feltéve, hogy `T`-nél alacsonyabb teljesítmények is megengedettek!

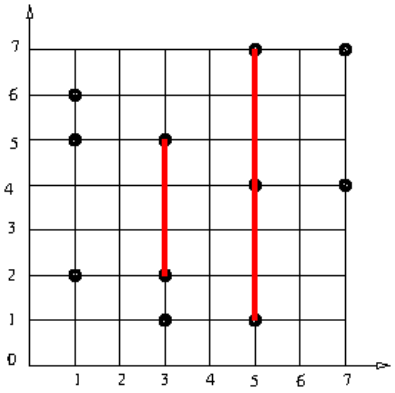
Példa

Bemenet

```
11
1 2
1 5
7 4
3 2
3 5
5 1
5 4
3 1
5 7
7 7
1 6
```

Kimenet

```
3
3
```



7. Feladat3: https://cses.fi/problemset/task/1673

High Score

Tekintsük azt a játékot, amelyben a játéktábla `n` szobát tartalmaz, amelyeket között `m` alagút köt össze. A szobákat az `1, ..., n` számokkal azonosítjuk. Szobák között sétálva pontokat gyűjthetsz, kezdetben 0 pontod van. Minden alagútra adott, hogy ha azon áthaladsz, akkor a pontszámodhoz hozzáadódik az alagút pontszáma. Minden alagút pontszáma egész szám, ami lehet negatív is. A játék kezdetén az 1 sorszámú szobában vagy és az `n` sorszámúba kell eljutni.
Minden alagúton akárhányszor áthaladhatsz.

Készíts programot, amely az elérhető legnagyobb pontszámot!

Bemenet

A `standard bemenet` első sorában a szobák száma $(1 \leq N \leq 2\,500)$ és az alagutak száma $(1 \leq M \leq 5\,000)$ van. A következő M sorban egy-egy alagút adata van, három egész szám: `a b x`, ami azt jelenti, hogy az `a` és a `b` szoba között olyan alagút van, amin `a`-ből `b`-be lehet átmenni és ezért `x` pontot kapsz.
Biztosan el lehet jutni az 1 szobából az `n` sorszámúba.

Kimenet

A `standard kimenetre` első sorába a legnagyobb elérhető pontszám értékét kell írni! Ha akármilyen nagy pontszám elérhető, akkor a -1 értéket kell kiírni! .

Példa

Bemenet	Kimenet
4 5 1 2 3 2 4 -1 1 3 -2 3 4 7 1 4 4	5

Korlátok

Időlimit: 1 mp.
Memórialimit: 512MB